

Scripts Documentation

This document provides an overview of utility scripts used in the project for analyzing and plotting results from `elliott`.

1. `create_datasets.py`

The `create_datasets.py` script is responsible for parsing the different results file of `elliott` creating for each threshold an unique csv file with all the metrics of the different models and integrating the CR (Candidate Ratio) metric.

How It Works Internally

1. **Dataset Integration:** It uses the `create_integrated_dataset()` function, taking raw performance metrics from a defined directory (e.g., `result/target_t_0_05/performance`). It consolidates these individual result files into a single consolidated CSV dataset, saved in the `data/` folder.
2. **Merging CR (Component Return):** It reads the `CR_verification.csv` file and merges this data into the previously created dataset using the `add_cr_column()` function.
- 3.

Data Verification: At the end of execution, the script loads the resulting dataset using `Pandas`, verifies the presence of the `CR` column, and prints a summary including the first few rows and the count of valid/missing `CR` values.

2. `plot_generator.py`

The `plot_generator.py` script is a data visualization tool that generates 2D and 3D scatter/line plots. It visualizes the trade-offs between two or three different metrics (e.g., `ApproxSeverity` vs. `nDCG`, or `ApproxSeverity` vs. `nDCG` vs. `Gini`) for various recommendation models. It integrates statistical significance testing results (from `filtered_t_test` files) directly on the plots with asterisk markers. In particular the `t` test values are the one that compare the current model with the KNN with the same similarity and number of neighbours.

How It Works Internally

1. **Data Loading & Metric Extraction:**
 - Iterates through the given data directory (parsing files starting with `target_t_`) and strictly filters by the user-requested neighbor counts (`nn`).

- Dynamically parses 2 or 3 input metrics (`x_metric`, `y_metric`, `z_metric`). Also handles special cases, such as extracting target threshold values as a pseudo-metric called `ApproxSeverity`.
2. **Naming & Grouping Logic:**
 - Aligns and curates model names based on algorithms, similarity metrics (`sim`), and sampling strategies (`strat`).
 - Identical to other scripts, it customizes ambiguous FairANN model names into clear concepts like `Minhashing` or `LSHRandomProjection` depending on the used similarity.
 3. **Statistical Significance Overlays:**
 - Scans the `filtered_t_test` directory to find corresponding t-test results for each threshold.
 - For every data point representing a model that demonstrated statistically significant differences for the specified metrics, the script sets an internal `_sig` Boolean flag to True.
 4. **Plot Generation:**
 - **2D Plots:** Uses `matplotlib` to render multi-line scatter plots. Statistical significance points are redundantly highlighted with a black asterisk (*).
 - **3D Plots:** Uses `plotly` to render interactive interactive HTML 3D plots. Significant datapoints are marked with opaque black diamonds.
 - Subplotting logic automatically maps data based on models, similarity measures, or across multiple neighbor (`nn`) configurations to keep graphs readable.

Command-Line Flags

The script uses standard `argparse` flags for execution:

- `--data_dir`: Path to the directory containing the source CSV files. (*Default: `data`*)
- `--metrics` (**Required**): A space-separated list specifying 2 or 3 metrics to plot against each other. If 2 metrics are provided, a 2D line plot is created. If 3 are provided, an interactive 3D Plotly graph is rendered.
 - *Example:* `--metrics Recall nDCGRendle2020` or `--metrics Recall nDCGRendle2020 Gini`
- `--split`: A boolean flag. If activated, it separates plots by the underlying recommender model type into `User` and `Item` categories.
- `--split_sim`: A prioritized boolean flag that further isolates models into four sub-categories: User vs. Item algorithms, subdivided by Cosine vs. Jaccard similarities.
- `--combined_nn`: A boolean flag. When provided, the script merges the plots of all considered neighbor values (`nn` = 50, 100, 250) into unified graphical figures with multiple subplots side-by-side, instead of generating completely disparate files.

3. multi_obj_analisys.py

The `multi_obj_analisys.py` script performs a multi-objective analysis by plotting the Pareto frontier (in 2D or 3D). It evaluates models across multiple performance metrics simultaneously identifying the set of optimal models where no model can improve one metric without degrading another. It can also compute and save the Hypervolume index for these frontiers.

How It Works Internally

1. **Data loading:** Instantiates a `ParetoPlotter3D` object and loads performance results from all `.csv` files found within the specified `data_folder`.
2. **Algorithmic Naming Customizations:** Iterates through the loaded dataset and differentiates models like `KNNfairness` by correctly appending their pre-processing/sampling techniques (`preposp`) to their `Algorithm` name so they form distinct entities on the chart.
3. **Pareto Plotting:** Executes the `plot_pareto()` method of the plotter. By observing the requested validation metrics and optimization directions, it isolates the Pareto-optimal points. It supports visually splitting the graphs by similarity measure or threshold, showing/hiding sub-optimal points, and re-scaling specific dimensions logarithmically.
4. **Hypervolume Computation:** If requested, the script can calculate the Hypervolume (a metric capturing both the convergence and diversity of the Pareto front) based on the supplied directions and saves it out into a `_hv.csv` dataset.

Command-Line Flags

The script supports an extensive set of `argparse` flags to highly configure the analysis:

- **--data_folder (Required):** The base folder containing the source `.csv` dataset files to analyze.
- **--metrics (Required):** A space-separated list of exactly 2 or 3 metric names to use as the axes of your plot (e.g., `--metrics nDCGRendle2020 Recall Gini`).
- **--directions (Required):** A space-separated list specifying the optimization directions for the corresponding metrics provided above. Use `max` for metrics to maximize (like Recall) and `min` for metrics to minimize (like Gini). (*Must be equal in length to --metrics*).
- **--output_file:** Base name/path for the exported plots/HTML files. (*Default: `pareto_plot`*)
- **--log_metrics:** List of specific metrics to scale logarithmically in the plot visualization to ease the inspection of skewed data (e.g., `--log_metrics Gini`).

- `--split_threshold`: A boolean flag. If activated, separate Pareto plot files are generated for each unique target threshold (t) found in the data.
- `--split_sim`: A boolean flag. If activated, it separates plots by the underlying similarity computation method (Cosine vs. Jaccard).
- `--compute_hypervolume`: A boolean flag. When enabled, it calculates the hypervolume indexes of the resulting Pareto fronts and writes them to a CSV file (appended with `_hv.csv`).
- `--hv_split_threshold`: A boolean flag. Similar to `--compute_hypervolume`, but computes and exports the hypervolume measures grouped separately for each independent target threshold value.
- `--only_pareto`: A boolean visual flag. Selecting this hides all non-optimal internal datapoints, limiting the visualization purely to models sitting precisely on the Pareto optimal frontier.